RISC-V指令集乱序处理器设计

毕业论文开题报告

学院：计算机科学与技术

指导老师：薛睿

学号：220111012

姓名：王靳

报告提纲

1. 课题背景
2. 研究目的与意义
3. 国内外研究现状
4. 现有研究的不足
5. 主要研究内容
6. 研究方案
7. 进度安排
8. 预期目标

一、课题背景

RISC-V 指令集架构的蓬勃发展

- 2010 年由加州大学伯克利分校提出，采用**开放源代码许可**
- 设计理念：**精简、模块化、可扩展**
- RISC-V International 已拥有超过 **4000 家**会员单位
- 应用覆盖从嵌入式微控制器到高性能服务器的完整生态

> 打破传统商业指令集架构（x86、ARM）的授权壁垒，为自主可控芯片提供可行技术路线

一、课题背景

产业界的广泛采纳

- **乐鑫科技** ESP32-C3/C6 系列 — 物联网终端设备
- **NVIDIA** 已部署超过 **10 亿颗** RISC-V 核心用于片上控制
- **工信部** 2023 年成立 RISC-V 工作委员会，推动规模化应用
- **国家战略**：2025 年八部门联合发布指导意见，鼓励推广使用开源 RISC-V 芯片

一、课题背景

乱序执行 — 提升指令级并行度的关键

- 1967 年 Tomasulo 提出动态调度算法，是现代高性能处理器的核心支柱
- 在不违反程序语义的前提下，**重新排列指令的执行顺序**
- 有效利用功能单元的空闲周期，显著提升流水线吞吐率

二、研究目的与意义

课题目标

> 使用 **Chisel** 硬件描述语言，设计并实现一个支持**乱序执行**的 RISC-V 处理器， 并构建包含自主设计 IP 核的完整 **SoC** 系统。

二、研究目的与意义

🔧 服务信创产业

掌握高性能处理器微架构的自主设计能力，为国产处理器研发提供技术储备

🧩 丰富 CHISEL 生态

使用 Chisel 重写 SDRAM、QSPI 等外设 IP 核，接入 Diplomacy 参数协商框架

📚 填补教学空白

定位于教学级与工业级之间，复杂度适中的单发射乱序处理器参考设计

🔗 完整验证闭环

从UVM单模块验证到回归测试

三、国内外研究现状

乱序执行关键技术

技术	要点
Tomasulo 算法	保留站实现操作数动态转发与分布式等待
寄存器重命名	消除 WAR / WAW 伪依赖，仅保留 RAW 真依赖
重排序缓冲区	维护程序顺序，支持精确异常与推测执行
分支预测	BHT、gshare、RAS 等，准确率 > 95%

三、国内外研究现状

代表性开源 RISC-V 处理器

项目	来源	特点
BOOM	UC Berkeley	超标量乱序，Chisel 实现，RV64GC
Rocket	UC Berkeley	顺序执行五级流水，SoC 生成框架
香山（昆明湖）	中科院计算所	6 发射、256 项 ROB，高性能
蜂鸟 E203	核芯互联	超低功耗嵌入式
玄铁系列	平头哥半导体	商业产品，C906/C910/C920 覆盖低功耗到高性能

三、国内外研究现状

硬件描述语言与验证方法

- **Chisel**：基于 Scala 的硬件构造语言
  - 与 Verilog 同属 RTL 层级，经过 circt 编译生成 Verilog
  - 更强的类型安全、参数化生成、模块复用
  - 软件工程方法应用于硬件设计
- **多层次验证体系**：
  - 模块级：UVM 受约束随机验证
  - 系统级：riscv-arch-test 指令集合规性
  - 差分测试：Difftest — RTL 与参考模型逐指令对比

四、现有研究的不足

1. 教学级与工业级乱序处理器之间存在显著断层

BOOM/香山复杂度极高（~15 万行），简单五级流水又未涉及乱序机制，**中间缺少进阶参考设计**

2. CHISEL 生态下可复用 IP 核积累不足

SoC 外设 IP 核仍以 Verilog/VHDL 为主，需通过 BlackBox 封装，**无法发挥 Chisel 与 Diplomacy 优势**：参数化、模块复用、类型安全

五、主要研究内容

总体架构

1. 多周期 SoC → 2. 乱序执行微架构 → 3. 系统软件适配

渐进式开发：先实现功能完备的多周期顺序 SoC，再逐步引入乱序执行模块，最终运行操作系统

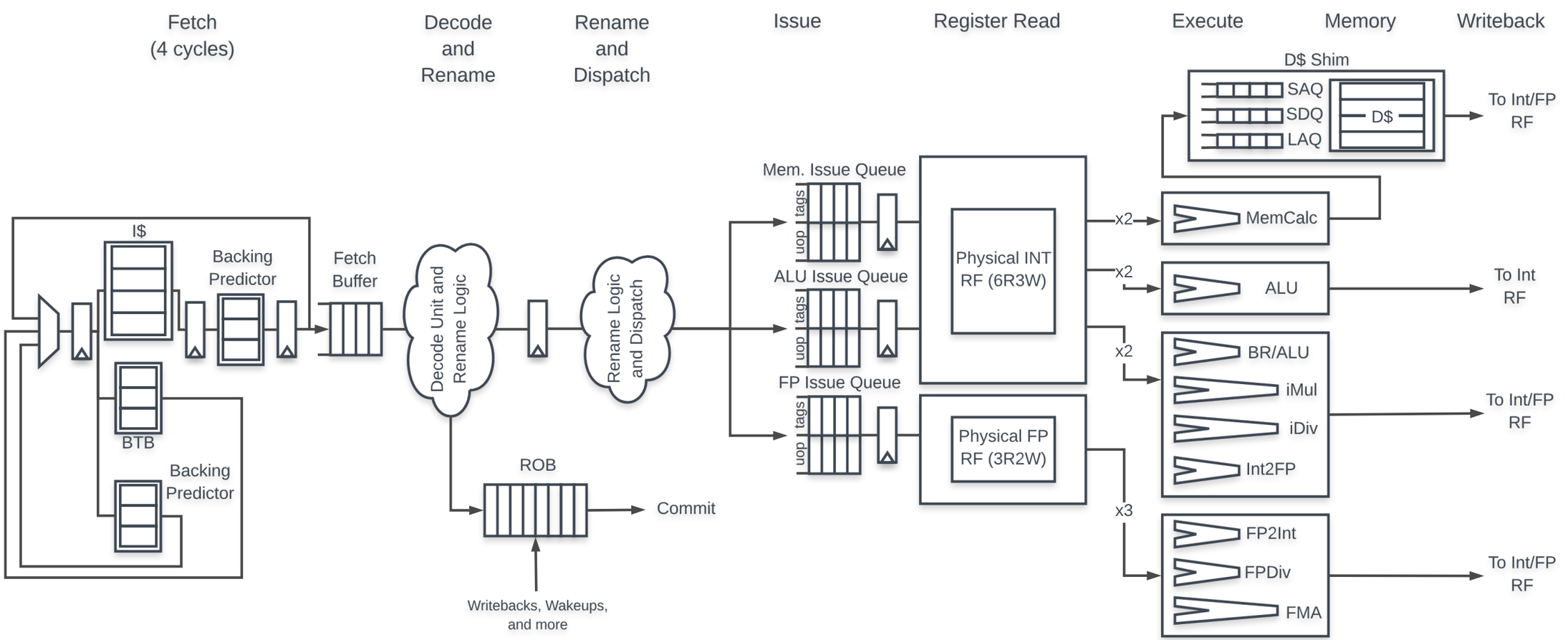
五、研究内容

1. 多周期基础处理器 (RV32I)

- **指令集**：RV32I 全部指令（算术逻辑、加载存储、分支跳转、系统指令）
- **数据通路**：FSM 驱动的多周期数据通路，灵活适配不同延迟的总线与外设
- **SoC 集成**：AXI4 总线、SDRAM 控制器、UART 等基础外设

五、研究内容

2. 乱序执行微架构



五、研究内容

3. 系统软件适配与操作系统引导

- **特权架构**：M-mode / S-mode CSR、中断异常处理、Sv32 虚拟内存
- **中断控制器**：CLINT（定时器/软件中断）+ PLIC（外部中断路由）

> **验证目标：**
> ✅ 首要目标 — 运行 **xv6-riscv** 教学操作系统
> 🚀 进阶目标 — 适配 OpenSBI + U-Boot，引导 **Linux** 内核启动

六、研究方案

阶段一：基础处理器搭建与验证

- 配置 Chisel/Scala + Verilator 仿真 + RISC-V 交叉编译工具链
- 实现 RV32I 多周期处理器（FSM 驱动控制逻辑）
- 集成 AXI4 总线、SRAM、UART

> **产出：** RTL 代码 + riscv-arch-test 100% 通过 + Coremark 裸机运行

六、研究方案

阶段二：乱序执行核心模块设计

自底向上的增量式开发：

1. 物理寄存器堆 + 空闲列表 (Free List)
2. 寄存器重命名模块 (RAT + 快照机制)
3. 发射队列 + 选择逻辑
4. ROB + 顺序提交逻辑
5. 功能单元 (ALU / Branch Unit / LSU)
6. 分支预测器 (BHT + BTB)
7. 前端与后端整合

六、研究方案

验证策略：多层次验证体系

层次	方法	目标
指令级	riscv-arch-test	指令集合规性 100%
架构级	Difftest (Spike/NEMU)	运行 C 语言测试用例，逐指令对比
系统级	运行 xv6 / Linux	全栈功能正确性

七、进度安排

2026.01 — 2026.02

基础处理器设计与 SoC 集成

RV32I 多周期处理器 → AXI4 SoC → FPGA 验证 → riscv-arch-test 100%

2026.02 — 2026.03

乱序执行微架构设计与性能调优

核心模块 RTL 实现 → Difftest 验证 → Coremark/Dhrystone 性能评估

2026.03 — 2026.04

系统集成与操作系统引导

特权架构 → CLINT/PLIC → xv6 运行 → Linux 引导（进阶）→ 论文撰写

八、预期目标

目标项	预期成果
指令集兼容性	riscv-arch-test **100% 通过**
功能验证	Difftest 对比 **无差异**
操作系统	xv6 **稳定运行**，通过 usertests
进阶目标	Linux 内核引导至用户空间 Shell
IP 核生态	Chisel 实现的 SDRAM / QSPI 可复用 IP 核

谢谢！